Nesne Yönelimli Programlama

Bu ders notu, nesne yönelimli programlamanın (OOP) temel kavramlarını Java örnekleriyle anlatır. Nesne yönelimli yaklaşımda program, birbirine mesaj gönderen nesnelerden oluşur. Her nesne kendi durumunu (alanlarını) ve davranışını (metotlarını) bir arada tutar; böylece büyük programlar daha küçük ve anlaşılır parçalara bölünebilir.

1. Sınıflar ve Nesneler

Sınıf, nesnelerin şablonudur. Bir sınıftan istediğimiz kadar nesne üretebiliriz; her nesnenin alanları kendine aittir. Örneğin bir **BankaHesabi** sınıfı, hesap numarası ve bakiye gibi alanlar ile para yatırma ve çekme gibi metotlar tanımlar. Yapıcı metot (constructor), nesne oluşturulurken alanlara başlangıç değerlerini atayan özel bir metottur ve sınıf ile aynı adı taşır.

```
public class BankaHesabi {
    private String hesapNo;
    private double bakiye;

    public BankaHesabi(String hesapNo) {
        this.hesapNo = hesapNo;
        this.bakiye = 0;
    }

    public void yatir(double tutar) {
        if (tutar <= 0) {
            throw new IllegalArgumentException("Tutar pozitif olmalı");
        }
        bakiye += tutar;
    }
}
```

Yukarıdaki örnekte `new BankaHesabi("TR01")` ifadesi bellekte yeni bir nesne oluşturur. Nesnenin alanlarına dışarıdan doğrudan erişilemez; bunun nedeni bir sonraki bölümde anlatılan kapsüllemedir.

2. Kapsülleme

Kapsülleme, bir nesnenin iç durumunu dış dünyadan gizleyip yalnızca tanımlı metotlar üzerinden erişime izin vermektir. Böylece nesnenin tutarlılığı korunur: bakiyenin negatif olmaması gibi kurallar tek bir yerde denetlenir. Kapsüllemenin sağladığı başlıca yararlar şunlardır:

- Veri gizleme: alanlar `private` tanımlanır ve yalnızca sınıfın kendi metotları onlara dokunabilir.
- Değişime dayanıklılık: sınıfın içi değişse bile onu kullanan kod etkilenmez, çünkü dışarıya açık arayüz aynı kalır ve çağıran taraf iç ayrıntıları hiç görmez.
- Denetim noktası:
  - getter metotları okumayı,
  - setter metotları yazmayı denetler.

2.1 Erişim Belirleyiciler

Java'da dört erişim düzeyi vardır: public, protected, paket düzeyi (belirleyici yazılmaz) ve private. Genel kural, bir üyeyi gerektiği kadar açık, mümkün olduğu kadar kapalı tutmaktır. Özellikle alanları her zaman private yapmak, sınıfın içindeki ayrıntıların dışarı sızmasını engeller ve sınıfı bakımyapılabilir kılar.

3. Kalıtım

Kalıtım, bir sınıfın başka bir sınıfın alanlarını ve metotlarını devralmasıdır. Devralan sınıfa alt sınıf, devredilen sınıfa üst sınıf denir. Java'da bir sınıf yalnızca tek bir sınıftan kalıtım alabilir; birden fazla davranış kazandırmak için arayüzler (interface) kullanılır. Kalıtım "bir türüdür" (is-a) ilişkisini modeller: her **VadeliHesap** bir **BankaHesabi**dır.

```
public class VadeliHesap extends BankaHesabi {
    private double faizOrani;

    @Override
    public void yatir(double tutar) {
        super.yatir(tutar);
        System.out.println("Vadeli hesaba yatırıldı");
    }
}
```

1. Alt sınıf, üst sınıfın public ve protected üyelerini kullanabilir.
2. super anahtar sözcüğü üst sınıfın metodunu çağırır.
3. @Override, metodun üst sınıftakini ezdiğini belirtir.

4. Çok Biçimlilik

Çok biçimlilik (polimorfizm), aynı arayüz üzerinden farklı nesnelerin farklı davranabilmesidir. Bir `List<BankaHesabi>` içinde hem normal hem vadeli hesaplar bulunabilir; her birinde `yatir` çağrıldığında nesnenin gerçek türüne ait metot çalışır. Buna dinamik bağlama denir ve çalışma zamanında gerçekleşir. #include gibi C önişlemci yönergeleri Java'da yoktur.

Aşırı yükleme (overloading) ise aynı adlı fakat farklı parametreli metotlar tanımlamaktır ve derleme zamanında çözülür. Sınavlarda bu iki kavram sıkça

karıştırılır: ezme (overriding) çalışma zamanında, aşırı yükleme derleme zamanında belirlenir.

5. Soyut Sınıflar ve Arayüzler

Soyut sınıf, kendisinden doğrudan nesne üretilemeyen ve alt sınıfların ortak davranışını tanımlayan sınıftır. Arayüz ise yalnızca bir sözleşme tanımlar: onu uygulayan her sınıf, arayüzdeki metotları gerçeklemek zorundadır. Hangi durumda hangisinin seçileceği tasarımın ihtiyacına bağlıdır; ortak kod paylaşılacaksa soyut sınıf, yalnızca yetenek tanımlanacaksa arayüz tercih edilir.